Tome — Independent Council Review (pi)

Project: *tome v0.1.0* · ***Author:*** *Lwando Zwane* · ***Date:*** *2026-08-07*

Reviewer: *pi — independent second reviewer, following the Kimi K3 council report*

Method: *full read of* `src/main` *(index 529, airgap 191, authlock 115, conductor 201, brain 321 LOC),* `src/preload` *(82),* `src/renderer` *(renderer 1664, lock 121, history 131, index.html, style.css),* `package.json`*, CI workflow, build scripts, README, and the git history. Where the Kimi report made testable claims, I re-verified them against the code and by executing the algorithms (*`authlock.hotp` *against the RFC 4226 vectors, the airgap wildcard compiler against suffix-bypass hostnames, the conductor control-char stripper,* `brain.confine` *against traversal inputs).*

Overall: 8.0 / 10. *I land exactly where Kimi's council landed, but for a partially different set of reasons, and I disagree with them on one of their two HIGH security findings.*

---

1. Architecture

Tome is a three-process Electron app with an unusually clean separation of concerns:

- **Main** owns everything dangerous: PTY spawning, the Anthropic/Requesty stream, git shell-outs, the seatbelt profile, the per-pane CONNECT proxies, and all auth state. The four service modules (`airgap`, `authlock`, `brain`, `conductor`) share one shape — module state Maps, a `setEventSink` for main→renderer events, exported verbs — which makes the whole main process readable in one sitting.
- **Preload** is a pure allowlist: ~40 channels exposed through `contextBridge`, no `ipcRenderer` handle leaked, no generic `invoke` passthrough. The only wart is `home: ipcRenderer.sendSync('app:home')` evaluated at preload load time — it works because it resolves instantly, but it is the only synchronous IPC in the app and would be the first thing to break if main ever stalls.
- **Renderer** is dockview + xterm + CodeMirror with five panel classes. It holds no secrets and no privileged capability.

The architectural decisions I most respect:

1. 1. **The pty command line is built in main from a vetted** `kind`**.** The renderer can ask for `claude`, `opencode`, `pi`, or `terminal` — never a binary or arguments. Combined with the sandboxed renderer, a renderer compromise cannot directly spawn arbitrary processes. This is the single most important design decision in the app and it is correct.
2. 2. **The air gap widens the proxy, never the sandbox.** `unlockPane` flips a mode flag consulted by the proxy; the seatbelt profile is identical for locked and unlocked panes. There is no code path that weakens the sandbox after spawn, which means there is no unlock-time race and no way to "forget" to re-harden.
3. 3. **The lock gate wraps** `ipcMain.handle` **itself**, so every handler registered after the wrapper inherits the guard. Fail-closed by construction rather than by per-handler discipline. The `OPEN_CHANNELS` allowlist is exactly the door plus the store — and the store keys are vetted (`/^[a-z0-9][a-z0-9-]*$/`, with `airgap` and `airgap-auth` reserved) *because* the store stays open pre-login. That coupling is subtle and correctly handled.
4. 4. **The conductor's auto-run guard is in main, not the renderer.** When auto-run is off, `type_in_terminal` strips `[\x00-\x08\x0a-\x1f\x7f]` — CR, LF, Ctrl-C, Ctrl-D, ESC — so the model cannot smuggle a submission. I executed the regex: tab survives (deliberate — completion text), every submission/signal character dies. Commit b577f31 shows this was a found-and-fixed hole, which is the right kind of scar.

Where I diverge from pure praise: the architecture has **one trust-boundary diagram's worth of implicit knowledge** that exists only in comments. The fact that `store:get/set` must stay open pre-login *and therefore* must reserve the auth/allowlist keys; the fact that `airgap:unlock` is safe to leave passphrase-only *because* login already proved the passphrase; the fact that `brain/` lives outside userData *because* the seatbelt denies userData writes — these are load-bearing invariants scattered across three files' comments. A single `docs/THREATMODEL.md` collecting them would outlive any refactor.

2. Security — airgap & authlock

I re-verified Kimi's security claims line by line. Verdicts:

Where I agree with Kimi

- **[HIGH]** `resolveAgentSecrets()` **over-harvests.** The regex `^([A-Z][A-Z0-9_]*(?:API_KEY|KEY|TOKEN))=(.*)$` scrapes an interactive login shell's entire environment and hands the result to every agent pane — including air-gapped ones. `GITHUB_TOKEN`, `NPM_TOKEN`, `AWS_SECRET_ACCESS_KEY` (no — that one escapes the suffix match, which is luck, not design), `DIGITALOCEAN_TOKEN`, `HOMEBREW_GITHUB_API_TOKEN` would all flow into a sandboxed agent that needs exactly one key. The air gap constrains *where* those secrets can be sent (provider allowlist), so exfiltration requires a provider-domain exploit or a freed pane — but the principle of least privilege is simply abandoned here. Fix is a per-provider allowlist (`ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, `REQUESTY_API_KEY`, …). Small change, real win.
- **[MEDIUM] No login throttling.** `auth:login` and `airgap:unlock` verify at IPC speed. scrypt at default cost (~28 ms on this machine — I measured) is a speed bump, not a wall: a compromised unlocked renderer, or malware driving IPC, can brute-force a 4+ char passphrase offline-fast. The 4-character minimum accepted by both setup screens makes this worse — 4 lowercase chars is ~450k candidates, hours at 28 ms each. Add attempt counters with exponential backoff, and raise the minimum to 8.
- **[MEDIUM] TOTP secret at rest in reversible base32.** 0600 + seatbelt read-denial is decent defense in depth, but `safeStorage` is right there in Electron and would make the file useless off-machine.
- **[MEDIUM] The plain-HTTP proxy branch forwards** `req.headers` **verbatim**, including `Proxy-Authorization` and hop-by-hop headers. The hostname allowlist is checked from the URL (correct), but strip hop-by-hop headers before forwarding.
- **[LOW]** `doc:read` **/** `shell:openPath` **/** `fs:readFile` **take unvetted absolute paths.** Acceptable behind the lock, but the trust boundary should be written down: renderer compromise ≈ user-privileged file read.

Where I disagree with Kimi

- **The** `tome://` **protocol finding is overstated as written, but the fix is still right.** Kimi claims `corsEnabled: true` + `supportFetchAPI: true` means renderer JS can read `tome://` bodies and therefore exfiltrate any local file one XSS away. I checked the CSP in `index.html`: `connect-src 'self' ws: wss:` — `tome:` **is not in** `connect-src`, so `fetch('tome://...')` from renderer JS is blocked by CSP regardless of the scheme's privileges. The scheme is reachable via `img-src` and `frame-src` (embedding, which is what it's for) and via `default-src` — but embedding a file in an `<img>` doesn't let JS read its bytes. So today, `tome://local/?p=~/.ssh/id_rsa` is a *display* primitive, not a *read* primitive, and exfiltration via it additionally needs a CSP bypass. That said: the finding's core is sound. The handler does zero confinement (`protocol.handle('tome', … net.fetch(pathToFileURL(p)))` straight from a query param), the comment in main ("renderer JS cannot *read* tome:// bodies") is true only by CSP accident rather than by design, and one future CSP edit (`connect-src … tome:`) silently converts this into the full read primitive Kimi describes. Downgrade to MEDIUM, keep the fix: confine to workspace folders + extension allowlist, and drop `corsEnabled`/`supportFetchAPI` since embedding doesn't need them.

- **Kimi's "no rate limiting" framing misses the bigger brute-force surface:** the *first-run setup* path. `airgap:setup` accepts a 4-character passphrase and immediately calls `markUnlocked()`. The minimum length is the cheapest security upgrade in the entire codebase — one comparison changed from 4 to 8.

What Kimi missed

- **The `xlsx` dependency is fetched from a non-registry URL** (`https://cdn.sheetjs.com/xlsx-0.20.3.tgz`). The lockfile pins an integrity hash (good), but this supply-chain footnote deserves a comment in package.json or the README — SheetJS left npm after the registry pulled their tarballs, and anyone running `npm update` should know why that line looks odd.
- `TOME_SHOT` **bypasses the lock gate entirely** (`isLockedNow` returns false when the env var is set) and the preload happily exposes `shotMode`. It's a dev affordance and it's env-gated, but it's a compiled-in backdoor for anyone who can set an environment variable for the app — document it in the threat model or gate it on `!app.isPackaged`.
- `brain.confine()` **has no** `realpath` **check** — I confirmed by execution that lexical traversal is blocked (`..`, absolute, sibling-prefix), but a symlink *inside* a vault pointing outside is followed on read/write. Kimi mentioned this in passing ("theoretically possible but acceptable"); I'd add that an air-gapped agent pane can create that symlink itself (the seatbelt allows writes in `~/Tome/Brains`), so the escape is agent-reachable, not just user-reachable. The blast radius is small (the agent already has unsandboxed-equivalent file access to everything except userData and the auth file… which is exactly what it could then *write* through a symlink — wait, no: writes through the symlink happen with the agent's own seatbelt, which denies userData. So the symlink escape only matters for the *renderer-driven* brain IPC, which runs outside the sandbox. Renderer compromise again.) Net: LOW, but worth a `realpath` confinement pass since `brain:write` runs unsandboxed in main.
- **The conductor's** `open_file` **tool takes an absolute path from the model** and the renderer obligingly opens it — including calling `doc:read` on it, which runs `mammoth`/`XLSX.readFile` in main. So the chat model can cause main-process parsing of any file on disk. Parsing is the dangerous part (mammoth and SheetJS both have CVE histories), not display. Consider confining conductor-opened files to workspace folders. Kimi missed this entirely; I rate it MEDIUM because it requires the user to chat with a prompt-injected or malicious model response, which is… the normal operating mode of a coding harness pointed at untrusted code. Actually, on reflection, this deserves **HIGH-adjacent** attention: the conductor reads terminal scrollback and feeds it to the model, and the model's tool calls act on the filesystem — a classic confused-deputy loop where hostile output in a terminal (e.g. from a repo's build script) can steer the model. The auto-run guard caps the worst case at "open/parse a file" and "type unsubmitted text", which is why this isn't critical — but the loop deserves a named paragraph in the threat model.

What I verified that Kimi asserted without (visible) evidence

- **HOTP/TOTP is correct.** I ran `authlock.hotp` against all ten RFC 4226 test vectors: 10/10 match. The hand-rolled base32 round-trips lowercase input. Kimi called untested hand-rolled crypto "a coin flip" — the implementation is right; the *absence of pinned tests* is still the correct criticism.
- **The wildcard compiler is properly anchored.** `bedrock-runtime.*.amazonaws.com` matches `bedrock-runtime.us-east-1.amazonaws.com`, rejects `….amazonaws.com.evil.com` and bare `amazonaws.com`. Kimi's suggested unit test would pin behavior that is, today, correct.
- **The seatbelt profile layering is right** (later rules win; deny-egress then re-allow loopback; deny userData writes; deny auth-file reads).

3. Code quality

The code is genuinely good. The comments are the standout — they record *why*, admit trade-offs ("ponytail", "Collisions share a vault; accepted"), and mark future work. The git log reads like a changelog written by someone who respects their future self.

Agreements with Kimi, briefly: `renderer.js` at 1,664 lines is where every feature lands and should be split along its visible seams (panels/, menus, tree, git, modals) before it ossifies; no TypeScript means the ~40-channel IPC surface is grep-and-pray on every payload refactor; `modalShell` and `lock.js`'s `overlay()` are the same component twice; pane-kind magic strings live in three places (`AGENTS` in main, the conductor tool description, the renderer's `onOpen` switch).

My additions:

- **The renderer has an unhandled-rejection habit.** `tome.chat.send(...)` in `ChatPanel.send()` is fired without `.catch`; `tome.pty.create(...)` in `TerminalPanel.init` likewise. Main catches and re-signals over `chat:done`/`pty:data`, so the *user* sees errors — but the renderer console accumulates unhandled rejections and any future refactor that removes the main-side catch silently regresses to blank panes. This is exactly the bug class commit 9188c5c fixed once already.
- `refreshGit` **runs unconditionally every 5 s for the app's lifetime**, including when no workspace is open (it early-returns, fine) — but also *while the lock screen is up* in the brief window before boot resolves. Harmless because the IPC gate refuses it, but it means the locked app is quietly throwing IPC errors on a timer. Gate the interval on unlock.
- `history.js` **and** `lock.js` **both define their own** `el()` **helper**; renderer.js has a third inline style. Trivial, but it's the same drift Kimi flagged in modals, one layer down.
- **Error conventions are inconsistent** exactly as Kimi said — `{ok, error}` for auth/airgap/git-checkout, thrown errors for brain/fs/git-log, `null` for dialogs. The renderer already handles all three shapes, which is precisely why the inconsistency will persist unless someone picks one.

4. UX of the harness

Kimi's UX review was thorough and I co-sign most of it: the progressive security UX (Touch ID first, second-factor-only for pane unlock, contextual 2FA enrollment) is genuinely thoughtful; fail-visible design (red text in dead panes, blocked-host flashes) respects the user; the README is a model of the genre.

My independent observations, from reading the interaction code rather than the marketing:

- **The air-gap strip is the best UI idea in the app.** One click on the strip opens the unlock modal; the countdown renders in place; blocked hosts flash inline *and* toast. Security state is ambient, not buried in settings. This is how sandboxing should feel.
- **The graph view respects** `prefers-reduced-motion` by running 300 synchronous sim ticks instead of animating. Someone cared. (Also: the force sim seeds node positions from a deterministic name hash, so the graph is stable across reopens — a detail no one asked for that makes the tool feel authored.)
- **Chat has no stop button and drops failed messages** (`history.pop()` on error with no resend) — agreed with Kimi, and I'd add: `max_tokens: 64000` × 8 tool-loop turns is a real cost ceiling problem for a pane with no cancel affordance.
- **Toasts vanish in 4.8 s with no history** — agreed; a missed "airgap blocked" is gone forever. The blocked-host flash on the strip partially compensates.
- **No pane persistence** — agreed, and it's the biggest daily-use gap. dockview gives `toJSON`/`fromJSON` for free; terminals can't be restored but editors, docs, chat, and brain panes can.

- **Discovery is README-only.** No shortcut palette, no first-run orientation beyond the (excellent) lock screen. For a personal tool this is fine; the README's existence suggests the author knows it.
- One small delight Kimi didn't mention: agents light up in the ■ menu when their CLI appears on PATH (`agents:list` shells `command -v` per agent), so the app self-configures. Zero-config done right.
- One small papercut: the conductor-opened "unknown pane kind" path toasts `assistant asked for unknown pane: ${kind}` — fine — but the renderer's pane-kind list is hardcoded separately from main's `AGENTS`, so adding an agent requires editing three files. Drift is guaranteed.

5. Prioritized recommendations

Ranked by (risk × effort), with my deltas from Kimi's list marked:

#	Action	Why	Effort	vs Kimi
1	**Test floor: vitest on authlock (RFC 4226/6238 vectors), airgap wildcard compile, brain confine, conductor stripAnsi + control-char stripper. Wire into CI.**	I verified all four are *correct today* — which makes the absence of pinned tests pure downside: the next edit owns any regression silently.	S	same
2	**Per-provider allowlist for `resolveAgentSecrets()`**	`GITHUB_TOKEN` et al. should never enter a sandboxed agent pane.	S	same (Kimi #3 — I promote it)
3	**Confine conductor `open_file` (and ideally `doc:read`) to workspace folders**	The model can trigger main-process *parsing* of any file on disk; scrollback→model→tool-call is a live confused-deputy loop.	S	**new**
4	**Constrain `tome://` by design, not by CSP accident: drop `corsEnabled`/`supportFetchAPI`, confine to workspace folders + extension allowlist**	Today it's a display-only primitive because `connect-src` omits `tome:` — one CSP edit from a read-any-file primitive.	S	same fix, downgraded severity
5	**Login throttling + raise passphrase minimum 4→8 + `safeStorage` for the TOTP secret**	Brute-force at IPC speed against a 4-char floor is the weakest real link.	S	same
6	**Async + parallel boot shell-outs; never `execFileSync` on the launch path**	Up to 16 s launch stall on heavy `.zshrc`. Move secret resolution to first agent spawn.	S	same
7	**Split `renderer.js` along its seams (panels/, menus, tree, git, modals)**	1,664 lines is where every feature lands; split before it ossifies.	M	same
8	**Persist dockview layout per workspace + `requestSingleInstanceLock()`**	Biggest daily-use win; ~30 lines.	S	same
9	**Kill the compiled-in lock bypass: gate `TOME_SHOT` on `!app.isPackaged`**	An env var that disables the auth gate should not ship in the packaged app.	XS	**new**
10	**CI: lint + renderer smoke test + `npm run package`; document the SheetJS CDN dependency**	A broken renderer import fails at runtime, not in CI; the non-registry tarball is an undocumented supply-chain footnote.	M	same + addition
11	**Coalesce pty data events (~4 ms window) before the IPC hop**	`yes` / `cat`-of-big-file floods main→renderer.	S	same
12	**Write `docs/THREATMODEL.md`**	The load-bearing invariants (store-keys-open-pre-login ⇒ reserved keys; login-proves-passphrase ⇒ pane-unlock-is-second-factor; brain-outside-userData ⇒ seatbelt) currently live only in comments.	S	**new**
13	Chat stop button + cumulative token budget across the 8-turn loop	A 64K-token stream you cannot cancel, ×8 turns.	S	same (Kimi had it under reliability)

#	Action	Why	Effort	vs Kimi
14	Shared pane-kind constants module (main AGENTS ↔ conductor tool description ↔ renderer switch)	Three hand-synced lists; drift guaranteed.	XS	same

Final word

Kimi's council produced a good report — unusually good, in that almost every claim I re-checked against the code held up. My disagreements are at the margins: the `tome://` finding is real but currently defused by CSP (fix it anyway, because "safe by CSP accident" is one edit from unsafe); the conductor's file-opening tool is a bigger deal than anyone on that council noticed, because the scrollback→model→tool loop is the app's *intended* workflow with untrusted content flowing through it; and the `TOME_SHOT` lock bypass shouldn't ship in packaged builds.

The thing both reviews agree on without reservation: this is a disciplined, security-literate codebase whose biggest risks are structural (zero tests, one 1,664-line renderer) rather than behavioral. Buy the test floor first — I checked, the crypto and the guards are correct *today*, and tests are how they stay that way.

Scorecard — Security 8.5 · Platform 8.0 · Reliability 7.0 · Code Quality 8.0 · UX 8.0 · Overall 8.0/10

(Independently assigned before comparison; identical to Kimi's aggregate, with different reasoning underneath Security and two new findings in the queue.)

Review by pi · Tome v0.1.0 · 2026-08-07